Indian Institute of Technology Bhilai
DSL351/DSP352: Big Data Analytics
Practice Questions (Classes 3–20)

1. **(Pseudo-code)** You are given a dataset of weather records in the format `(stationID: String, timestamp: Long, temperature: Double)`. You need to find the maximum temperature for each station per year, but the requirements specify that the values arriving at the reducer for each station must already be sorted by `timestamp` in descending order.

   Write the MapReduce pseudo-code/logic for a **Secondary Sort** pattern. Specifically, describe:

   a) The structure of the Composite Key.

   b) The logic for the Custom Partitioner.

   c) The logic for the Grouping Comparator.

2. **(Structured Streaming)** Complete the code to ingest data from Kafka, calculate the count of events in 10-minute **sliding windows** (sliding every 5 minutes), and handle late data up to 20 minutes using **watermarking**.

```
val streamDf = spark.readStream
    .format("kafka")
    .option("kafka.bootstrap.servers", "localhost:9092")
    .option("subscribe", "events_topic")
    .load()

val processedDf = streamDf
    .selectExpr("CAST(value AS STRING)", "timestamp")
    // TODO: Add watermark
    .____________________________________
    // TODO: Group by sliding window
    .groupBy(window(col("timestamp"), "___________", "___________"))
    .count()

val query = processedDf.writeStream
    .outputMode("___________")
    .format("console")
    .start()
```

3. **(Data Skew)** Explain the concept of **"Salting"** to handle data skew during a Join operation between a large skewed table (Table A) and a smaller table (Table B). Provide the pseudo-code logic for how you would modify the join keys in Table A and how you would prepare Table B to ensure a correct join.

4. **(Performance Tuning)** An iterative machine learning algorithm accesses a large DataFrame `trainingData` twenty times in a loop. The cluster has limited memory but very fast SSDs.

a) Between `MEMORY_ONLY`, `MEMORY_AND_DISK`, and `DISK_ONLY`, which `StorageLevel` would you recommend? Justify your choice based on the "re-computation vs. I/O" trade-off.

b) What is the difference between `cache()` and `persist(StorageLevel.MEMORY_ONLY)`?

5. **(Catalyst – UDF Impact)** A developer uses a Python UDF to filter data: `df.filter(my_python ...`. Explain why Catalyst treats this UDF as a "black box" and how this prevents optimizations like **Constant Folding** and **Predicate Pushdown** to the data source (e.g., Parquet).

6. **(Catalyst – Boolean Simplification)** A dynamically generated query from a BI tool produces the following filter: `WHERE (dept = 'HR' AND salary > 50000) OR (dept = 'HR' AND bonus > 5000)`. Show how the Catalyst optimizer simplifies this logical expression using the **Distributive Law** to reduce redundant checks.

7. (10 points) **(Catalyst – Common Subexpression Elimination)** Consider the following DataFrame transformation:

```
df.withColumn("total", col("price") * col("tax"))
  .withColumn("final", (col("price") * col("tax")) - col("discount"))
```

   Explain how Catalyst identifies the redundant calculation `price * tax` and how it optimizes the execution plan to avoid calculating it twice.

8. **(Data Sources – Handling Malformed Data)** You are building a streaming/batch job to ingest a directory of JSON files. Occasionally, upstream systems write malformed, unparseable strings into these files. By default, this might cause the Spark job to throw a `JsonParseException` and fail, or silently drop data.

   You are required to build a robust pipeline that:

   1. Prevents the job from failing when it encounters bad JSON lines.
   2. Captures the original malformed JSON strings into a specific column named `_corrupt_record`.
   3. Separates the records into two different DataFrames: one for valid records, and one "dead-letter" DataFrame containing only the corrupted records for alerting.

   Write the Scala DataFrame code snippet to achieve this.

9. **(Data Sources – Parallel JDBC Reads)** You need to ingest a massive `transactions` table (500 million rows) from a relational database (e.g., PostgreSQL) into Spark using the JDBC data source.

   a) If you execute `spark.read.jdbc(url, "transactions", properties)`, what is the primary performance bottleneck regarding how Spark fetches the data?

b) Provide the DataFrame reader code (including the specific `.option()` configurations) to ensure Spark reads this table in parallel using 20 concurrent tasks. Assume the table has an indexed numeric column `txn_id` ranging from 1 to 500,000,000.

10. **(Spark SQL – Window Functions)** A temporary view `employees` has columns `emp_id`, `dept_id`, and `salary`. Write the Spark SQL query to find the `emp_id`s of the top 2 highest-paid employees in each department.

```sql
SELECT emp_id, dept_id, salary
FROM (
    SELECT emp_id, dept_id, salary,
           -- TODO: Apply dense rank over department ordered by salary descending
           ____________________________________________________ AS rank
    FROM employees
) ranked
WHERE ____________________
```

11. **(DataFrame API – Anti Joins & Null Handling)** You have two DataFrames: `users` (`user_id`, `name`) and `orders` (`order_id`, `user_id`, `amount`). Complete the code to find users who have never placed an order, and create a DataFrame with their `name` and a `default_amount` of 0.0.

```scala
val noOrdersDf = users
    // TODO: Perform a left anti join
    .join(orders, Seq("user_id"), "_________________")
    // TODO: Add default_amount column literal
    .withColumn("default_amount", _________(0.0))
    .select("name", "default_amount")
```